Software Architecture and Process Models
INTE 22293- Academic Year 2024/2025
Session 2: Structure for the Software- Good Design





University of Kelaniya, Sri Lanka



Dr. Dilani Wickramaarachchi

PhD, MSc., BSc. (Special)

Senior Lecturer

Department of Industrial Management, Faculty of Science

E-Mail – dilani@kln.ac.lk

Session Overview

- **Topics you have already discussed:**
  - Software process models
- Characteristics of a good design
- Software Architecture vs Software Design
- Software Architecture vs Design patterns
- What is the importance of software architecture?



Characteristics of a good design

- **High Cohesion**
- General definition of cohesion
  - “the action or act of forming a united whole‘”
  - "the state of sticking together, or
  - “being in close agreement and working well together".
- Cohesion in Software development:
  - Software is divided into different modules.
  - These modules are developed independently
  - Modules are capable of performing some specific functionality.
  - For the software to be efficient,
    - these modules must have high cohesion.



Characteristics of a good design

- **Low Coupling**
- General definition of coupling:
  - “as a thing that joins together two objects”
- Coupling in Software development:
  - Coupling is related to the connection between two modules
  - The measure of the degree of the interdependency of two modules on each other is known as coupling



Simple Analogy:

- Think of a kitchen. A chef (high cohesion) does all the cooking-related tasks :- chopping, seasoning, plating.
- They don't need to know how the dishwasher works internally; they just hand dirty plates through a small, well-defined opening (low coupling) and the dishwashing station handles the rest.



Software Architecture and Software Design

- **Software Architecture:** is a plan that gives enough detail to produce a software design
- **Software Design:** is a plan that gives enough detail to implement software

- **System Analysis:** An investigation of the problem (rather than how a solution is defined)
- **System Design:** Emphasizes a conceptual solution that fulfills the requirements.
  - Designs are implemented in a programming language



Software Architecture and Software Design Pattern

- Architecture comes in Designing phase and
- Design Patterns comes in Building phase.
- Architectural pattern is like a blueprint and
- Design pattern is actual implementation.
- Architecture is base which everything else adhere to and
- Design pattern is a way to structure classes to solve common problems



Why do we need Software Architecture?

- any software have a significant impact on the business of an organization.
- one of the main reason for any software failure can be the selection of wrong software architecture patterns.
- application development, without a formal architecture in place what would happen?
  - they end up with codes that lack clear roles, responsibilities, and relationships with one another.
  - planning an architecture pattern helps in analyzing risks beforehand and avoids any adverse impact on the business.



Architect vs. Developer vs. Tech Lead

Dimension	Developer	Tech Lead	Software Architect
Primary focus	Implement features	Team execution & code quality	System structure & long-term integrity
Time horizon	Sprint / task	Sprint / release	Months to years
Typical decisions	Function/class design	Task breakdown, code review standards	System boundaries, tech stack, integration patterns
Key skill	Coding proficiency	Mentoring + delivery	Trade-off analysis + communication
Audience	Team members	Team + PM	Executives, teams, other architects

Core Responsibilities of a Software Architect

System Decomposition

Break the problem into components/services with clear boundaries and responsibilities.

Trade-off Analysis

Weigh performance, cost, scalability, and time-to-market against each other.

Stakeholder Communication

Translate business goals into technical direction, and technical constraints into business language.

Quality Attribute Ownership

Own non-functional requirements: security, scalability, availability, maintainability.

Decision Documentation

Record key decisions and rationale so future teams understand ‘why,’ not just ‘what.’

Technical Roadmap

Guide the evolution of the system as requirements and scale change over time.

Quality Attributes (“the -ilities”)

Functional requirements say what the system does.
Quality attributes say how well it does it , and they drive most architecture decisions.

Performance
Response time & throughput under load

Scalability
Ability to handle growth in users/data

Availability
Uptime and fault tolerance

Security
Protection of data and access control

Maintainability
Ease of modifying and extending the system

Interoperability
Ability to integrate with other systems

Evaluating an Architecture Before You Build It

Architects stress-test designs on paper using structured walkthroughs — the best-known is ATAM (Architecture Trade-off Analysis Method).

1

Identify scenarios

Concrete situations: “10x traffic spike during a sale”

2

Map to quality attributes

Which -ility does each scenario stress?

3

Analyze architectural approaches

How does the current design respond?

4

Surface risks & trade-offs

Where does the design break, and what's the cost to fix it?

What is an Architectural Pattern?

- An outline that allows you to express and define a structural schema for all kinds of software systems.
- It's a reusable solution that provides a predefined set of subsystems, roles, and responsibilities, including the rules and roadmap for defining relationships among them.
- It helps us to address various software engineering concerns such as ***performance limitations, high availability, minimizing business risk***, etc.
- An architectural pattern is a rough image or blueprint of the system.



Different types of software architecture patterns

- Monolithic Architecture
- Multi Layered Architecture
- MVC Architecture
- Even Driven Architecture
- Client Server Architecture
- Microservice architecture



What is the importance of software architecture pattern?

- Defining Basic Characteristics of an Application:
- Maintaining Quality and Efficiency:
- Providing Agility:
- Problem Solving:
- Enhancing Productivity:



For each architecture:

- Architecture pattern:
- Usage:
- Shortcomings:
- Any example applications:



Monolithic Architecture

A **monolithic application** means the entire system is built as one large application.



End of the Session 2



Go back to Session Overview